

Day 5: Git Workflow & HTML/CSS Foundations

TRACK	TOPIC	TARGET
Full-Stack Web Development	Git Kata, Rebase, Conventional Commits, Semantic HTML5 & Responsive CSS	Production-Grade & WAVE Compliant

Overview: Day 5 bridges foundational programming with professional team collaboration and frontend architecture. You will learn to execute enterprise Git workflows (branching strategies, resolving complex merge conflicts, interactive rebasing/squashing, and imperative commit conventions), construct semantic, accessible HTML5 layouts that pass strict WCAG standards, master the CSS layout trinity (Box Model, Flexbox, Grid), and build responsive, mobile-first components and CSS-only interactive navigation systems.

1. Enterprise Git Branching Models

In high-velocity engineering environments, developers never commit directly to `main`. Structured branch naming creates clear visibility for automated CI/CD pipelines, code reviews, and issue tracking:

Branch Prefix	Purpose & Scope	Example Branch Name
feat/	New capability, UI component, or API endpoint	feat/tower-progress-card
fix/	Bug fix resolving an issue or defect	fix/meter-contrast-ratio
chore/	Maintenance, build scripts, linter or config updates	chore/update-eslint-rules
docs/	Documentation updates, READMEs, architectural notes	docs/day5-git-workflow
refactor/	Code reorganization without changing external behavior	refactor/css-custom-properties

2. Git Merge vs Git Rebase & Resolving Conflicts

Integrating concurrent work from team members can be done via **Merge** (creates a 2-parent merge commit) or **Rebase** (replays your commits linearly on top of the target branch):

INTENTIONAL MERGE CONFLICT RESOLUTION (EXERCISE 1)

```
// 1. Concurrent change on 'main':  
Tower A: Structural Audit In Progress  
Tower B: Foundation Drilling Started  
  
// 2. Conflicting change on 'feat/tower-updates':  
Tower A: Superstructure Started  
Tower B: Excavation In Progress  
Tower C: Planned  
  
// 3. Git Conflict Markers Generated:  
<<<<<< HEAD  
Tower A: Structural Audit In Progress  
Tower B: Foundation Drilling Started  
=====  
Tower A: Superstructure Started  
Tower B: Excavation In Progress  
Tower C: Planned  
>>>>>> feat/tower-updates  
  
// 4. Manually resolved and combined:  
Tower A: Structural Audit & Superstructure Started  
Tower B: Foundation Drilling Started  
Tower C: Planned
```

CLEAN COMMIT LOG GRAPH AFTER RESOLUTION & SQUASHING:

```
* 5eb521a docs: append audit verification timestamp and auditor details  
* 539e7d1 merge: resolve tower status conflict between audit and progress updates  
|\n| * 430ffa6 feat: update Tower A progress to superstructure  
* | ccaae8 fix: record structural audit on Tower A  
|/  
* 8b49cdc feat: initialize tower status log
```

3. Commit Message Conventions & Pull Request Etiquette

At NirmanIQ, all commit messages follow the **imperative mood** convention. A commit message must complete the sentence: *"If applied, this commit will..."*

- **Correct:** feat: add tower progress card component
- **Incorrect:** added tower progress card (past tense) or adds card (third person)
- **PR Protocol:** Open Draft PRs early; keep diffs under 400 lines; address all review feedback politely.

4. Semantic HTML5 & WCAG Accessibility (No Div Soup)

Using semantic elements instead of generic `<div>` tags provides native browser accessibility, keyboard support, and proper screen reader navigation without requiring complex JavaScript:

Semantic Element	Accessibility Landmark / Role	Why Not Generic <code><div></code> ?
<code><header></code>	Banner landmark	Screen readers announce page and article introductory regions.
<code><nav></code>	Navigation landmark	Allows blind users to skip directly to site navigation links.
<code><main></code>	Main content landmark	Only one per document; enables instant "skip to content" shortcuts.
<code><article></code>	Self-contained composition	Identifies reusable components like cards or report items.
<code><button></code>	Interactive control	Natively supports Tab focus and fires on Enter / Space keys.
<code><dl></code> , <code><dt></code> , <code><dd></code>	Description / metadata list	Semantic grouping for metric labels and values (Floors: 18/24).

WCAG Color Contrast Standards

To pass the automated WAVE accessibility checker, all foreground text must exceed the WCAG 4.5:1 (AA) or 7:1 (AAA) contrast ratios:

Status Badge	Background Color	Text Color	Contrast Ratio	Compliance Level
On Track (Green)	#dcfce7 (Light Green)	#14532d (Deep Forest Green)	8.8 : 1	WCAG AAA Passed
At Risk (Amber)	#fef3c7 (Light Amber)	#78350f (Deep Brown/Amber)	7.2 : 1	WCAG AAA Passed
Critical (Red)	#fee2e2 (Light Red)	#7f1d1d (Deep Crimson)	8.4 : 1	WCAG AAA Passed

5. The CSS Layout Trinity: Box Model, Flexbox & Grid

- Box Model:** Always set `box-sizing: border-box`; globally so padding and borders are contained within defined dimensions.
- Flexbox (1D):** Used for single-axis alignment (e.g. horizontal navbar links, card header title/badge alignment, action buttons).
- CSS Grid (2D):** Used for multi-column responsive dashboard cards (`grid-template-columns: repeat(3, 1fr)`) and metric key-value layouts.

6. Mobile-First Responsive Design (360px → 768px → 1024px)

Mobile-first CSS writes base rules for 360px screens first, using `min-width` media queries to progressively enhance the experience on larger viewports:

Breakpoint	Target Viewport	Layout Architecture
Base (< 768px)	Smartphones (360px – 480px)	Single-column vertical stack, full-width cards, collapsible drawer menu
Tablet (>= 768px)	Tablets & iPads (768px – 1023px)	Two-column card grid, horizontal inline flex navbar (hamburger hidden)
Desktop (>= 1024px)	Laptops & Monitors (1024px+)	Three-column card grid, max-width 1200px centered container

7. Pure CSS Accessible Hamburger Menu Architecture (Exercise 3)

Demonstrating UI interactivity without JavaScript by leveraging native browser behavior:

CSS-ONLY KEYBOARD-ACCESSIBLE TOGGLE PATTERN

```
<!-- HTML Structure -->
<input type="checkbox" id="nav-toggle" class="nav-toggle" aria-label="Toggle navigation">
<label for="nav-toggle" class="nav-toggle-label">
  <span class="hamburger-bar bar-top"></span>
  <span class="hamburger-bar bar-mid"></span>
  <span class="hamburger-bar bar-bot"></span>
</label>

<ul class="nav-menu">
  <li><a href="#projects" class="nav-link">Projects</a></li>
  <li><a href="#towers" class="nav-link">Towers</a></li>
</ul>

/* CSS Rules */
.nav-toggle { position: absolute; opacity: 0; width: 0; height: 0; }
.nav-toggle:focus-visible + .nav-toggle-label { outline: 3px solid #0ea5e9; }
.nav-toggle:checked ~ .nav-menu { max-height: 400px; padding: 1rem 1.25rem; }

@media (min-width: 768px) {
  .nav-toggle-label { display: none; }
  .nav-menu { position: static; flex-direction: row; max-height: none; }
}
```

ACCESSIBILITY VERIFICATION:

Tab focus active on hamburger checkbox -> Press Space key natively expands/collapses menu.
Zero JavaScript required. WAVE Accessibility evaluation: 0 Errors, 0 Contrast Alerts.

8. Deliverables & Friday Demo Summary

- **Exercise 1 (Git Kata):** Automated script testing branch divergence, intentional merge conflicts, manual conflict resolution, and commit squashing.
- **Exercise 2 (Tower Progress Card):** Semantic HTML card with floor counters, accessible progress bar, high-contrast badges, and responsive CSS.
- **Exercise 3 (Responsive Navbar):** Pure CSS collapsible hamburger menu, keyboard-navigable via Tab and Space.
- **Friday Demo (index.html):** Unified dashboard integrating navbar and multi-tower cards across 360px, 768px, and 1024px screen sizes.